

Procédure de calibration — Moteurs Feetech STS3215 avec LeRobot (HuggingFace)

Référence matérielle : Servomoteurs Feetech STS3215 (série STS, protocole 0) **Framework :** LeRobot (HuggingFace) — branche **main** **Contexte cible :** Bras robotique SO-101 (ou hardware compatible)

Prérequis

Matériel

- Moteurs Feetech STS3215 (7,4 V ou 12 V selon version)
- Adaptateur bus servo (ex. : WaveShare Bus Servo Adapter) relié via USB
- Alimentation adaptée : **5 V** pour les moteurs 7,4 V, **12 V** pour les moteurs 12 V
- Câbles 3 broches pour la connexion en chaîne (daisy-chain)

⚠ Attention : Ne jamais brancher une alimentation 12 V sur des moteurs 7,4 V, sous peine de les endommager définitivement.

Logiciel

```
# Cloner le dépôt LeRobot et installer les dépendances Feetech
git clone https://github.com/huggingface/lerobot.git
cd lerobot
pip install -e ".[feetech]"
```

Étape 1 — Identifier les ports USB

Chaque adaptateur bus servo est associé à un port série sur l'ordinateur. L'utilitaire **lerobot-find-port** permet de les identifier automatiquement.

1. Brancher l'adaptateur bus servo via USB (sans alimentation moteur nécessaire à cette étape).
2. Exécuter la commande :

```
lerobot-find-port
```

3. Suivre les instructions à l'écran : débrancher le câble USB lorsque demandé, puis appuyer sur **Entrée**.

Exemple de sortie :

```
Finding all available ports for the MotorBus.
['/dev/tty.usbmodem575E0032081', '/dev/tty.usbmodem575E0031751']
Remove the USB cable from your MotorsBus and press Enter when done.
```

The port of this MotorsBus is /dev/tty.usbmodem575E0032081
Reconnect the USB cable.

- **macOS** : /dev/tty.usbmodem*
- **Linux** : /dev/ttyUSB0 ou /dev/ttyACM0
- **Windows** : COM3, COM4, etc.

Répéter pour chaque adaptateur (bras leader et bras follower si applicable).

Étape 2 — Configuration des IDs et baudrates des moteurs

Chaque moteur doit posséder un **identifiant unique** (ID) sur le bus. Par défaut, tous les moteurs sortent d'usine avec l'ID 1. Il faut donc les configurer **un par un**, en les branchant individuellement à l'adaptateur.

Cette configuration est écrite en **mémoire EEPROM non-volatile** : elle n'est à réaliser qu'une seule fois par moteur.

Ordre de configuration recommandé (bras à 6 degrés de liberté)

Articulation	ID moteur
Shoulder Pan	1
Shoulder Lift	2
Elbow Flex	3
Wrist Flex	4
Wrist Roll	5
Gripper	6

Procédure (bras follower)

1. Connecter l'alimentation et le câble USB à la carte contrôleur.
2. Lancer le script de configuration :

```
lerobot-setup-motors \  
  --robot.type=so101_follower \  
  --robot.port=/dev/tty.usbmodem585A0076841
```

3. Le script demande de connecter **uniquement le moteur gripper** à la carte :

Connect the controller board to the 'gripper' motor only and press enter.

4. Brancher le câble 3 broches entre la carte et le moteur **gripper uniquement** (pas de daisy-chain à cette étape). Appuyer sur **Entrée**.
5. Le script configure automatiquement l'ID et le baudrate, puis confirme :

```
'gripper' motor id set to 6
```

6. Répéter pour chaque moteur dans l'ordre indiqué par le script (**wrist_roll**, **wrist_flex**, etc.) jusqu'au **shoulder_pan** (ID 1).

Si vous utilisez une carte Waveshare, vérifier que les deux jumpers sont positionnés sur le canal **B (USB)**.

Procédure (bras leader, si applicable)

```
lerobot-setup-motors \  
  --teleop.type=s0101_leader \  
  --teleop.port=/dev/tty.usbmodem575E0031751
```

Même procédure que pour le follower, moteur par moteur.

Configuration via API Python (alternative)

```
from lerobot.robots.so_follower import S0101Follower, S0101FollowerConfig  
  
config = S0101FollowerConfig(  
    port="/dev/tty.usbmodem585A0076841",  
    id="my_follower_arm",  
)  
follower = S0101Follower(config)  
follower.setup_motors()
```

Étape 3 — Calibration des positions articulaires

La calibration établit la correspondance entre les positions brutes des encodeurs moteurs et les angles réels des articulations. Elle définit trois paramètres par moteur :

- **Homing Offset** : décalage appliqué à l'encodeur pour que la position de référence du robot corresponde à la valeur 0 dans l'espace normalisé
- **Range Min** : position minimale de la plage utile, **exprimée après application du Homing Offset**
- **Range Max** : position maximale de la plage utile, **exprimée après application du Homing Offset**

Cette étape est **critique** pour la reproductibilité : un réseau de neurones entraîné sur un robot fonctionnera sur un autre robot uniquement si les deux ont été correctement calibrés.

Rôle crucial de la Phase 1 — Calcul du Homing Offset

L'encodeur du STS3215 est **absolu sur 12 bits** : il couvre la plage 0 à 4095, où les valeurs 4095 et 0 sont physiquement adjacentes (discontinuité de "wrap-around"). Une plage d'utilisation qui passerait par cette discontinuité (ex. 3000 → 4095 → 0 → 1000) est invalide pour LeRobot, qui suppose que `range_min < range_max`.

La Phase 1 résout ce problème : en plaçant le moteur au **milieu de la plage utile souhaitée** et en appuyant sur **Entrée**, LeRobot calcule un **Homing_Offset** qui recale cette position à la valeur 0. Après ce recalage, la plage utile devient symétrique (ex. -1000 à +1000) et la discontinuité wrap-around est garantie d'être en dehors de la plage utile.

⚠ La position choisie en Phase 1 doit être le milieu de la plage utile, pas une position quelconque de repos. Une erreur ici décale toute la plage et peut replacer la discontinuité à l'intérieur de la zone de travail.

Exemple concret :

Avant calibration (brut)	Après calibration (normalisé)
Position brute : 2000	→ 0 (zéro de référence)
Position brute : 3000	→ +1000
Position brute : 1000	→ -1000
Discontinuité à 4095/0	→ ±2048 (hors plage utile ✅)

Calibration du bras follower

1. S'assurer que les bras follower **et** leader sont tous deux connectés (USB + alimentation).
2. Lancer la commande :

```
lerobot-calibrate \
  --robot.type=so101_follower \
  --robot.port=/dev/tty.usbmodem58760431551 \
  --robot.id=my_follower_arm
```

3. Suivre les instructions interactives :

- **Phase 1 — Réglage du Homing Offset** : Placer manuellement chaque articulation au **milieu exact de sa plage utile souhaitée** (pas seulement dans une position commode). C'est cette étape qui calcule le **Homing_Offset** et déplace la discontinuité 4095 → 0 hors de la zone de travail. Appuyer sur **Entrée** une fois la position atteinte.
- **Phase 2 — Balayage de l'amplitude** : Faire parcourir à chaque articulation **toute son amplitude de mouvement** (de la butée minimale à la butée maximale). Le script enregistre `range_min` et `range_max` dans l'espace recalé par l'offset de la Phase 1.

4. Le fichier de calibration est automatiquement sauvegardé :

```
~/cache/huggingface/lerobot/calibration/robots/so101_follower/my_follower_arm.json
```

Exemple de contenu du fichier JSON généré :

```
{
  "shoulder_pan": { "homing_offset": -2000, "range_min": -1100,
"range_max": 1100 },
  "shoulder_lift": { "homing_offset": -512, "range_min": -900,
"range_max": 900 },
  "elbow_flex": { "homing_offset": 204, "range_min": -1000,
"range_max": 1000 },
  "wrist_flex": { "homing_offset": 1024, "range_min": -800,
"range_max": 800 },
  "wrist_roll": { "homing_offset": 0, "range_min": -2000,
"range_max": 2000 },
  "gripper": { "homing_offset": -50, "range_min": 0,
"range_max": 800 }
}
```

Calibration du bras leader (si applicable)

```
lerobot-calibrate \
  --teleop.type=so101_leader \
  --teleop.port=/dev/tty.usbmodem575E0031751 \
  --teleop.id=my_leader_arm
```

Même procédure : Phase 1 au milieu de la plage utile, puis balayage de l'amplitude complète.

Le fichier est sauvegardé dans :

```
~/cache/huggingface/lerobot/calibration/teleoperators/so101_leader/my_leader_arm.json
```

Calibration via API Python

```
from lerobot.motors.feetech import FeetechMotorsBus
from lerobot.motors import Motor, MotorNormMode

motor_bus = FeetechMotorsBus(
    port="/dev/ttyUSB0",
    motors={
        "shoulder_pan": Motor(1, "sts3215", MotorNormMode.RANGE_M100_100),
```

```

        "shoulder_lift": Motor(2, "sts3215", MotorNormMode.RANGE_M100_100),
        "elbow_flex":    Motor(3, "sts3215", MotorNormMode.RANGE_M100_100),
        "wrist_flex":    Motor(4, "sts3215", MotorNormMode.RANGE_M100_100),
        "wrist_roll":    Motor(5, "sts3215", MotorNormMode.RANGE_M100_100),
        "gripper":       Motor(6, "sts3215", MotorNormMode.RANGE_M100_100),
    },
)

motor_bus.connect()
motor_bus.disable_torque()

# Phase 1 : positionner chaque moteur au milieu de sa plage utile avant
d'appeler calibrate()
# Phase 2 : le script guidera ensuite le balayage de l'amplitude
motor_bus.calibrate()
motor_bus.disconnect()

```

Étape 4 — Vérification de la calibration

Après calibration, lancer une session de télé-opération pour vérifier que les positions sont cohérentes entre les deux bras :

```

lerobot-teleoperate \
  --robot.type=so101_follower \
  --robot.port=/dev/ttyACM0 \
  --robot.id=my_follower_arm \
  --teleop.type=so101_leader \
  --teleop.port=/dev/ttyACM1 \
  --teleop.id=my_leader_arm

```

Points de vérification :

- Les deux bras répondent de manière symétrique et sans dérive.
- Aucun moteur n'atteint ses butées mécaniques à des positions qui devraient être centrées.
- Les positions lues correspondent aux angles visuels observés.

Dépannage

Wrap-around $4095 \rightarrow 0$: la calibration place-t-elle la discontinuité en dehors de la plage utile ?

Oui, c'est précisément le rôle de la Phase 1. Si le moteur est bien positionné au milieu de sa plage utile lors de la Phase 1, le **Homing_Offset** calculé garantit que la discontinuité wrap-around ($4095 \rightarrow 0$) se retrouve à ± 2048 dans l'espace normalisé — soit à l'opposé de la zone de travail.

En revanche, si la Phase 1 est réalisée avec le moteur dans une position excentrée, la discontinuité peut se retrouver à l'intérieur de la plage utile, provoquant des sauts de position inexplicables lors du balayage ou de l'utilisation.

Le moteur n'est pas détecté

- Vérifier le câble 3 broches (connexion ferme des deux côtés).
- Vérifier l'alimentation (LED de la carte allumée ?).
- Sur Waveshare : jumpers en position **canal B**.
- Un seul moteur doit être connecté pendant la phase de configuration des IDs.

Erreur `Missing motor IDs`

```
RuntimeError: FeetechMotorsBus motor check failed: Missing motor IDs: 2
```

Cause probable : le moteur a été configuré avec un baudrate différent. Recommencer la configuration des IDs pour ce moteur individuellement.

Erreur `COMM_RX_CORRUPT` ou `Incorrect status packet`

- Vérifier la tension d'alimentation (5 V pour STS3215 7,4 V).
- Réduire la longueur des câbles ou utiliser des câbles de meilleure qualité.
- S'assurer qu'il n'y a pas de conflit d'ID sur le bus.

La calibration doit être refaite à chaque redémarrage

Vérifier que le fichier JSON de calibration est bien présent et lisible :

```
cat  
~/ .cache/huggingface/lerobot/calibration/robots/so101_follower/my_follower_  
arm.json
```

Si le fichier existe mais que la calibration est ignorée, vérifier que le paramètre `--robot.id` utilisé à l'exécution correspond bien au nom du fichier JSON.

Récapitulatif des commandes principales

Action	Commande
Trouver les ports USB	<code>lerobot-find-port</code>
Configurer IDs des moteurs (follower)	<code>lerobot-setup-motors --robot.type=so101_follower --robot.port=<PORT></code>
Configurer IDs des moteurs (leader)	<code>lerobot-setup-motors --teleop.type=so101_leader --teleop.port=<PORT></code>
Calibrer le follower	<code>lerobot-calibrate --robot.type=so101_follower --robot.port=<PORT> --robot.id=<ID></code>

Action	Commande
Calibrer le leader	<code>lerobot-calibrate --teleop.type=so101_leader --teleop.port=<PORT> --teleop.id=<ID></code>
Tester en télé-opération	<code>lerobot-teleoperate --robot.type=so101_follower ... --teleop.type=so101_leader ...</code>

Références

- Documentation officielle LeRobot SO-101 : <https://huggingface.co/docs/lerobot/so101>
- Documentation Feetech Motors : <https://www.mintlify.com/huggingface/lerobot/motors/feetech>
- Dépôt GitHub LeRobot : <https://github.com/huggingface/lerobot>
- Guide matériel SO-ARM100 : <https://github.com/TheRobotStudio/SO-ARM100>